Creampan

SMART CONTRACT AUDIT REPORT

IMMUNEBYTES

28 October 2023

Introduction **3**

About Creampan 3

About ImmuneBytes 3

Documentation Details **4**

Audit Goals **4**

Audit Process & Methodology **5**

Audit Details **6**

System Architecture **7**

Security Level References **8**

Severity Status References **9**

Severity Status **9**

Finding **10**

Critical Severity Issues **11**

High Severity Issues **13**

Medium severity issues **14**

Low severity issues **15**

Informational **19**

Concluding Remarks **20**

Disclaimer **20**

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About Creampan

Creampan provides a liquid staking solution that you can mint ptCRO and ytCRO with CRO or WCRO on Cronos chain. The ptCRO is "principal token" of CRO and ytCRO is "yield token" of CRO. The ptCRO is designed to be used freely while the CRO is staked. You can hold, stake, farming, or even sell the ptCRO token when the CRO is staked in the chain. The ytCRO holders will receive "daily CRO reward" of the staked CRO. By separating the principle and yield, both tokens maintain constant price regarding to price of CRO. Therefore, delegators receive the real CRO rewards daily, freely participate DeFi earing opportunities without impermanent loss, and able to hedge against price of CRO by selling their ptCRO and holding ytCRO.

Visit https://creampan.gitbook.io/creampan-finance to know more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting smart contract audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 205+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The team has provided the following doc for audit:

1. https://creampan.gitbook.io/creampan-finance

Audit Goals

The focus of the audit was to verify that the smart contract system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within each contract and within the system of contracts.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established smart contract best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include

    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

ImmuneBytes team has performed thorough testing of the project, starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is a structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors which included -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Details

Project Name	Creampan
Platform	EVM
Contract	PanToken.sol, BakeryToken.sol, MasterBaker.sol
Languages	Solidity
GitHub Link	https://github.com/creampanfi/Creampan-contract/
Commit	a79e44d3809c2d8421b28d38d3427a5f5a1ebe93
Platforms & Tools	Remix IDE, Truffle, VScode, Contract Library, Slither, SmartCheck, Fuzz

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

System Architecture

High-Level Architecture

The system consists of multiple smart contracts interacting with each other and external contracts like Uniswap for liquidity provision.

Key Components

1. PanToken: ERC-20 compliant token, used for staking and rewards.
2. MasterBaker: Manages the farming pools and calculates rewards.
3. PanBakery: ERC-20 compliant token, used for staking and rewards.

Data Flow

1. Users stake PanToken into the MasterBaker contract.
2. MasterBaker calculates rewards based on staking time and amount.
3. Users can claim rewards or unstake through MasterBaker.

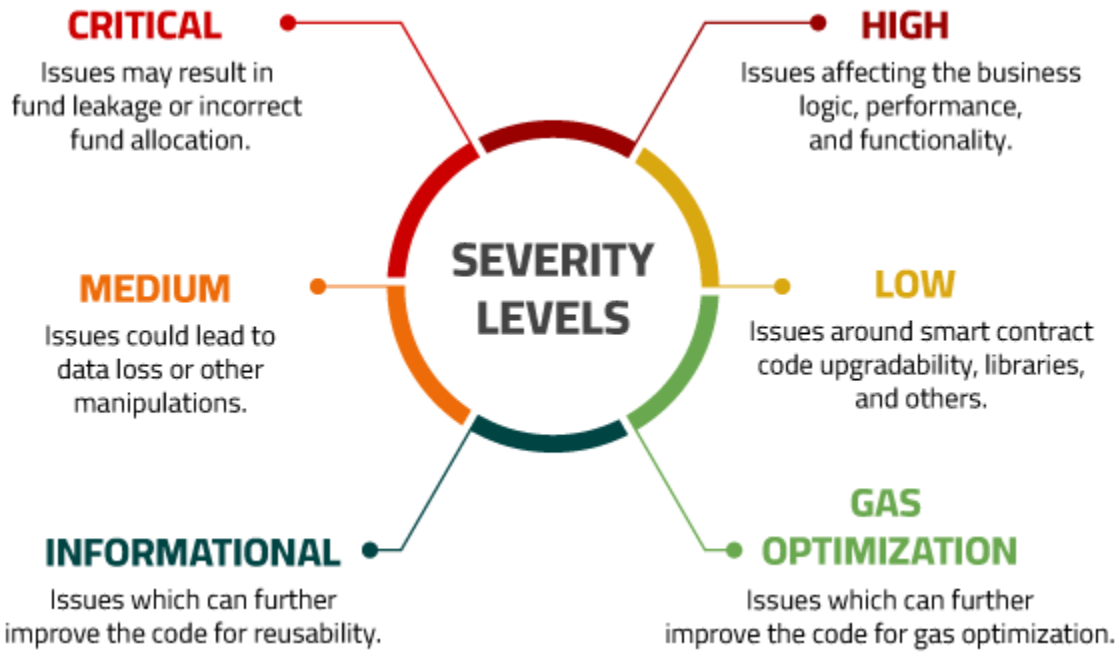
Security Mechanisms

- Role-based access control for admin functions
- Emergency withdrawal features for users
- Timelock mechanisms for governance decisions

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:



This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Severity Status References

Severity Status

Issues	Critical	High	Medium	Low	Informational	Gas
Open	-	-	-	-	-	-
Fixed	-	-	-	1	-	-
Redacted	-	-	-	-	-	-
Acknowledged	2	1	1	5	-	-

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Finding

#	Findings	Risk	Status
1.	Lack of Input Validation in PanToken constructor for initBlock	Critical	Acknowledged
2.	Reentrancy Attack Vector in emergencyWithdraw	Critical	Acknowledged
3.	Owner Can Arbitrarily Mint Tokens	High	Acknowledged
4.	Inadequate Visibility Settings and Abstract Intentions on updatePool	Medium	Acknowledged
5.	Insufficient Input Validation in setLimitEra	Low	Fixed
6.	Misleading Error Messages	Low	Acknowledged
7.	No Default Value Required	Low	Acknowledged
8.	Inconsistent Naming Conventions	Low	Acknowledged
9.	Commented Code	Low	Acknowledged
10.	Missing Constant or Setter Method	Low	Acknowledged

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

Issue	Lack of Input Validation in PanToken constructor for initBlock
Status	Acknowledged
Description	The PanToken constructor accepts an initial block number (_initBlock) without any validations. An improper setting could lead to DoS conditions in dependent functions such as MasterBakery.updatePanPerBlock().
Code Affected	PanToken Contract, constructor()
Impact	The lack of input validation for initBlock can lead to potential DoS attacks. Dependent functions might rely on this block number to be greater than or equal to the current block number, and incorrect values can disrupt the logic or lock the contract functions.
Recommendations	Implement input validation checks to ensure that initBlock is greater than or equal to the current block number.
Code Revised	constructor (uint256 _initBlock) public { require(_initBlock >= block.number, "Initial block must be >= current block"); initBlock = _initBlock; }
Developer Comment	The initialBlock number can be checked when developer deploys the contract. The constructor will only be called once. We think the risk can be controlled by checking carefully.
Auditor Comment	Acknowledged with no further action required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Issue	Reentrancy Attack Vector in emergencyWithdraw
Status	Acknowledged
Description	The emergencyWithdraw function in the MasterBaker contract is vulnerable to reentrancy attacks due to unsafe external calls.
Code Affected	MasterBaker Contract, emergencyWithdraw()
Impact	An attacker can exploit the reentrancy vulnerability to recursively call the emergencyWithdraw function, leading to unexpected state changes and potential loss of funds.
Recommendations	Use the Checks-Effects-Interactions pattern and consider employing the ReentrancyGuard modifier from OpenZeppelin.
Code Revised	// Using OpenZeppelin's ReentrancyGuard function emergencyWithdraw(uint256 _pid) public nonReentrant { ... }
Developer Comment	The emergencyWithdraw is exist as it is in Pancakeswap and Sushiswap contracts.The function didn't have a parameter to call another function outside the MasterBaker, PanToken, and PanBakery contract. But, it is ok to move "user.amout=0" and "user.rewardDebt=0" before safeTransfer is called. Commit: https://github.com/creampanfi/Creampan-contract/commit/a79e44d3809c2d8421b28d38d3427a5f5a1ebe93
Auditor Comment	Acknowledged with no further action required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

High Severity Issues

Issue	Owner Can Arbitrarily Mint Tokens
Status	Acknowledged
Description	The mint function in PanToken allows the owner to arbitrarily create tokens. This can lead to a loss of trust, as well as potential for abuse.
Code Affected	PanToken Contract, mint()
Impact	The arbitrary minting capability can severely compromise the token's integrity and value. It allows for unlimited token creation, which can lead to inflation and the devaluation of existing tokens, affecting all holders and stakeholders.
Recommendations	Implement a cap on the total supply and remove the minting function, or replace it with a more secure, community-governed mechanism.
Code Revised	uint256 public constant MAX_SUPPLY = 1000000 * 10 ** 18; function mint(address _to, uint256 _amount) public onlyOwner { require(totalSupply().add(_amount) <= MAX_SUPPLY, "Exceeds max supply"); _mint(_to, _amount); }
Developer Comment	The ownership will be transfered to MaskerBaker, which controlled the token supply. In the protocol litepaper, token schedule is well defined as decripted in the link. Link: https://creampan.gitbook.io/creampan-finance/pan/tokenomics The total supply is controlled by updatePool() and updatePanPerBlock() in MasterBaker. Design team designs to left it as the same.
Auditor Comment	Acknowledged with no further action required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Medium severity issues

Issue	Inadequate Visibility Settings and Abstract Intentions on updatePool
Status	Acknowledged
Description	The updatePool function in MasterBaker is publicly accessible without a clear reason. This can be potentially exploited and leads to ambiguous intentions.
Code Affected	MasterBaker Contract, updatePool()
Impact	Being publicly accessible, the updatePool function allows anyone to trigger it, potentially leading to gas inefficiencies or unexpected behavioral changes. It also introduces ambiguity in the contract's intentions, making it less secure and harder to audit.
Recommendations	Limit the visibility of the updatePool function to internal or restrict it to specific roles.
Code Revised	function updatePool(uint256 _pid) internal { ... }
Developer Comment	The updatePool is designed as public in intention. Everyone can toggle the function to update reward. The design is the same in PancakeSwap and Sushiswap without any change. Design team designs to left it as the same.
Auditor Comment	Acknowledged with no further action required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Low severity issues

Issue	Insufficient Input Validation in setLimitEra
Status	Resolved
Description	The function setLimitEra in contract MasterBaker doesn't validate the input parameter. This absence of input validation can potentially lead to undesirable states.
Code Affected	In contract MasterBaker, the function setLimitEra() is affected.
Impact	Without proper input validation, the contract is prone to edge cases where invalid or unintended values can be set. While this is a low-risk issue, it may lead to unexpected behavior that could be hard to debug later.
Recommendations	Implement input validation to check that the era being set meets expected criteria. This could be a range check or other logical conditions, depending on the use case.
Code Revised	-
Developer Comment	The limitEra range can be checked within a range like limitEra>0 and limitEra<=50. Commit: https://github.com/creampanfi/Creampan-contract/commit/a79e44d3809c2d8421b28d38d3427a5f5a1ebe93
Auditor Comment	-

Issue	Misleading Error Messages
Status	Acknowledged
Description	The contract uses unclear error messages like "deposit PAN by staking" and "withdraw: not good". These messages are not only unhelpful for debugging but can also be confusing for end-users.
Code Affected	In contract MasterBaker, the require statements in function deposit() and function withdraw() are affected.
Impact	Misleading or vague error messages make debugging more difficult and can result in a poor user experience. Users may not understand what went wrong or how to correct it, which can erode trust in the application.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Recommendations	Use more descriptive and precise error messages that offer clear guidance on the nature of the error and the steps needed to resolve it.
Code Revised	-
Developer Comment	The error message is the same as in PancakeSwap. Design team designs to left it as the same to reduce risk.
Auditor Comment	Acknowledged with no further action required.

Issue	No Default Value Required
Status	Acknowledged
Description	The variable totalAllocPoint is explicitly initialized to 0, which is redundant since Solidity automatically initializes uint256 variables to 0.
Code Affected	In contract MasterBaker, the variable totalAllocPoint is affected.
Impact	While this issue has minimal impact on gas costs, it does contribute to unnecessary lines of code, which can reduce readability and increase the likelihood of errors in the long run.
Recommendations	Remove the explicit initialization to adhere to best practices and improve code cleanliness.
Code Revised	-
Developer Comment	The totalAllocPoint initialization to 0 is the same as Pancakeswap and Sushiswap contracts. The totalAllocPoint will be assigned as 1000 in constructor. The risk is low. Design team designs to left it as the same to reduce risk.
Auditor Comment	Acknowledged with no further action required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Issue	Inconsistent Naming Conventions
Status	Acknowledged
Description	The function add in contract MasterBaker is vaguely named, which is inconsistent with best practices for code readability and self-documentation.
Code Affected	In contract MasterBaker, the function add() is affected.
Impact	Poorly named functions can severely impact the readability of the code. This can slow down development and increase the risk of errors, as developers may misunderstand the function's purpose. Additionally, the functionality can be misinterpreted by the users who interact with the contract via abi directly.
Recommendations	Use a more descriptive name for the function. For example, consider renaming add to addLPTokenPool to better reflect its functionality.
Code Revised	-
Developer Comment	Thanks for suggestion. The naming is the same as Pancakeswap and Sushiswap contracts. At the same time, the add() and set() functions can only access by "setter" as assigned though governance process. We don't expect user to access the function. The require() in the function will check the "setter". Design team designs to left it as the same to reduce risk.
Auditor Comment	Acknowledged with no further action required.

Issue	Commented Code
Status	Acknowledged
Description	Multiple instances of commented-out code exist within the smart contracts. While they don't affect the contract's behavior, they do compromise code quality by affecting readability and maintainability.
Code Affected	Multiple contracts, with lines such as //import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
Impact	Commented-out code can be confusing for developers who later work on this codebase. They might spend time trying to understand why the code was commented out, whether it is necessary for future use, or whether it hides a potential issue that was not properly addressed.
Recommendations	Remove all commented-out code unless it serves an explicit, documented purpose. If important for future reference, add a comment explaining why it is being kept.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Code Revised	-
Developer Comment	The commented codes for the "Ownable", "SafeMath", "IERC20", and "SafeERC20" are commented because the solidity version for the code are 0.8.0 solidity version. But the MasterBaker, PanToken, and PanBakery are written in 0.6.12. We use the right version of solidity for the contracts. We have removed the commented code. MasterBaker Commit: https://github.com/creampanfi/Creampan-contract/commit/a79e44d3809c2d8421b28d38d3427a5f5a1ebe93 PanBakery Commit: https://github.com/creampanfi/Creampan-contract/commit/00bf5afa2eedb8b252cfe706619406af571312d1 PanToken Commit: https://github.com/creampanfi/Creampan-contract/commit/ee1f4d5db8ea8b617f714743f7250857eebfd229
Auditor Comment	Acknowledged with no further action required.

Issue	Missing Constant or Setter Method
Status	Acknowledged
Description	The variables initBlock and blockPerDay are declared as public but are not declared as constants nor do they have setter methods. This raises questions about their mutability and the governance model surrounding these parameters.
Code Affected	In contract PanToken, the variables initBlock and blockPerDay are affected.
Impact	This design choice leaves the contract with a certain degree of inflexibility. If these values ever needed to be changed due to evolving business requirements or to react to an external event, such as a change in block time or other economic variables, there would be no way to do so without redeploying the contract. This could result in increased costs and operational complexity.
Recommendations	If these variables are intended to be immutable, explicitly declare them as constant to make this clear in the code. If they may need to be changed, implement a setter function that only the contract owner can call, and consider adding events to log any changes.
Code Revised	-
Developer Comment	The initBlock and blockPerDay is not expected to be changed after the deployment. We have add constant to declare for blockPerDay. For initBlock, it is assigned when contract deploymeny. We left it as public uint256 in contract. PanToken Commit: https://github.com/creampanfi/Creampan-contract/commit/ee1f4d5db8ea8b617f714743f7250857eebfd229
Auditor Comment	Acknowledged with no further action required.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Informational

No issues were found.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Concluding Remarks

While conducting the audits of Creampan, it was observed that the contracts contain Critical, High, Medium, and several Low severity issues. Our auditors suggest that the developers should resolve issues on priority before moving up with production. The recommendations given will improve the operations of the smart contract.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the Creampan platform or its product, nor is this audit investment advice.

Notes:

- Please make sure contracts deployed on the mainnet are the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES

AUDITS